

zAuth QR Sign-In for YouTube TV

Technical Documentation

Consolidated from the supplied Mintlify documentation export

sharvilgupta.mintlify.app

Contents

[How zAuth QR Sign-In Works: YouTube TV Technical Flow](#)
[YouTube TV Sign-In: QR-Based Authentication \(zAuth\)](#)
[POST /o/device/code — Initiate Device Authorization](#)
[Device Authorization Error Codes — zAuth API Reference](#)
[POST /token — Poll for Authorization and Refresh Tokens](#)
[zAuth Authentication Flow Reference for YouTube TV](#)
[Polling for zAuth Authorization Status on YouTube TV](#)
[Generating the zAuth QR Code for YouTube TV Sign-In](#)
[Code expired — loop and request a new one](#)
[zAuth Token Exchange and Session Management Reference](#)
[Decode without verification for display purposes only](#)
[Always verify the signature in production](#)
[Privacy and Data Access for YouTube on TV QR Sign-In](#)
[YouTube TV Token Lifecycle, Rotation, and Revocation](#)
[How zAuth Keeps Your YouTube on TV QR Sign-In Secure](#)
[Sign-In Complete: You Are Now Signed in to YouTube on TV](#)
[Step 3: Confirm Your Google Account on Your iPhone](#)
[Step 1: Display the YouTube TV QR Code on Your Screen](#)
[Step 2: Scan the YouTube TV QR Code with Your iPhone](#)
[YouTube TV zAuth: Supported Devices and Requirements](#)
[How to Switch or Add Google Accounts on YouTube on TV](#)
[YouTube TV QR Code Sign-In: Frequently Asked Questions](#)
[Troubleshoot iPhone QR Code Scanning for YouTube TV](#)
[YouTube TV Sign-In Error Codes and How to Fix Them](#)

How zAuth QR Sign-In Works: YouTube TV Technical Flow

Source: <https://sharvilgupta.mintlify.app/how-it-works>

A technical walkthrough of the zAuth QR sign-in flow: device code request, QR code rendering, iPhone confirmation, and OAuth token exchange.

zAuth is built on the **OAuth 2.0 Device Authorization Grant** ([RFC 8628](#)), a standard designed specifically for "limited-input devices" — hardware like TVs, game consoles, and set-top boxes that cannot safely accept credentials through a remote or on-screen keyboard. Rather than asking you to type a password on the TV, the flow delegates authentication entirely to a secondary device (your iPhone) that has a full browser, a secure keyboard, and direct access to your saved Google account. The TV simply waits, polls Google's servers, and picks up the resulting tokens once you've approved access on your phone.

The Three Actors

The TV

The **limited-input device** that initiates the flow. It requests a device code, renders the QR code on screen, and polls Google's token endpoint until authentication completes or the code expires.

Google's Auth Server

The **authorization authority** that issues device codes, hosts the user-facing sign-in page, validates account consent, and ultimately mints the `access_token` and `refresh_token` returned to the TV.

Your iPhone

The **secondary device** that handles all credential input. Your iPhone's Camera app decodes the QR code, opens Google's sign-in page in Safari or Chrome, and lets you securely select your account and grant consent — without the TV ever seeing your password.

Step-by-Step Flow

TV Requests a Device Code

The YouTube app on your TV sends a `POST` request to Google's device authorization endpoint:

```
POST /o/device/code
Content-Type: application/x-www-form-urlencoded
client_id=<YOUTUBE_CLIENT_ID>
&scope=https://www.googleapis.com/auth/youtube
```

This tells Google which application is requesting access and what permissions (scopes) it needs.

Google Returns the Device Code Payload

Google responds with a JSON object containing everything the TV needs to display the QR code and begin polling:

```
{
  "device_code": "AH-1Ng0bX...",
  "user_code": "GQHL-WJXZ",
  "verification_uri": "https://www.google.com/device",
  "verification_uri_complete": "https://www.google.com/device?user_code=GQHL-WJXZ",
  "expires_in": 1800,
  "interval": 5
}
```

Field	Purpose
<code>device_code</code>	Opaque token the TV uses when polling
<code>user_code</code>	Short human-readable code (shown as fallback text under the

Field	Purpose
	QR)
verification_uri	Manual sign-in URL if the user prefers to type
verification_uri_complete	The URL encoded into the QR code — includes <code>user_code</code> pre-filled
expires_in	Seconds until the code expires (1800 = 30 minutes)
interval	Minimum seconds between polling attempts (5 seconds)

TV Renders the QR Code and Begins Polling

The YouTube app encodes `verification_uri_complete` as a QR code and displays it on screen alongside the human-readable `user_code` as a fallback.

Simultaneously, the TV begins polling Google's token endpoint every 5 seconds:

```
POST /token
Content-Type: application/x-www-form-urlencoded
client_id=<YOUTUBE_CLIENT_ID>
&client_secret=<CLIENT_SECRET>
&device_code=AH-1Ng0bX...
&grant_type=urn:ietf:params:oauth:grant-type:device_code
```

While the user hasn't acted yet, Google returns `{"error": "authorization_pending"}` and the TV keeps polling.

User Scans the QR Code on Their iPhone

The user opens their iPhone Camera app and points it at the TV. The Camera app decodes `verification_uri_complete` and displays a banner notification. Tapping the banner opens `accounts.google.com` in the browser with the `user_code` already pre-filled.

The user selects their Google account (or signs in if not already), reviews the requested YouTube permissions, and taps **Allow**.

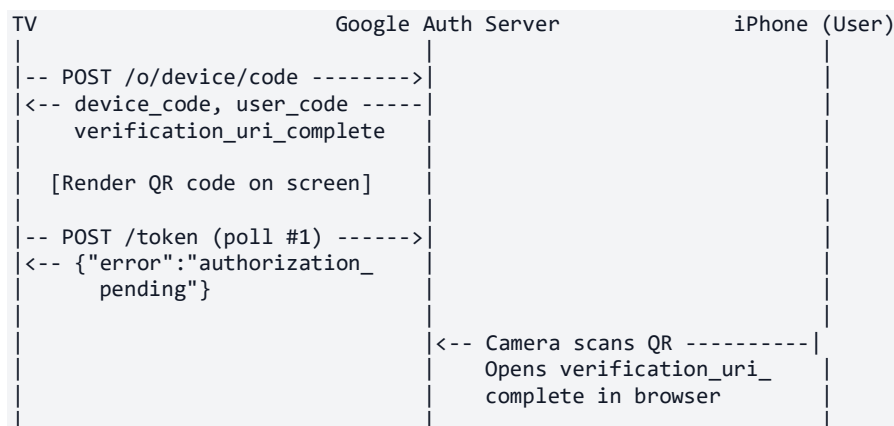
TV Receives Tokens — Authentication Complete

On the next polling cycle after the user taps Allow, Google's token endpoint returns a successful response:

```
{
  "access_token": "ya29.A0ARrdAM...",
  "expires_in": 3599,
  "refresh_token": "1//0eXk...",
  "scope": "https://www.googleapis.com/auth/youtube",
  "token_type": "Bearer"
}
```

The TV stores the `refresh_token` securely and uses the short-lived `access_token` to make YouTube API calls. The QR code is removed from the screen and the user is taken directly to their YouTube home feed.

Sequence Diagram



```

|                                     |<-- User selects account -----|
|                                     |    and taps Allow                |
|                                     |                                 |
|                                     |-- Grant recorded ----->|
|                                     |    (consent stored server-side)|
|                                     |                                 |
|-- POST /token (poll #N) ----->|
|<-- access_token,                  |
|    refresh_token                  |
|                                     |
| [User signed in – QR removed]    |

```

Key Properties

Important characteristics of the zAuth token flow:

- **Device code TTL:** Codes expire after **30 minutes** (`expires_in: 1800`). If the user doesn't scan in time, the TV automatically requests a fresh code and displays a new QR. A polling response of `expired_token` signals that the `device_code` TTL has been exceeded; the app must restart the flow by calling `POST /o/device/code` again.
- **Polling interval:** The TV polls no more frequently than every **5 seconds** (`interval: 5`). Polling too fast results in a `slow_down` error and a back-off penalty; the TV must increase its polling interval by at least 5 seconds before retrying.
- **User declined:** If the user taps **Deny** on the iPhone sign-in page, Google returns `access_denied`. The TV should stop polling and present the user with an option to try again.
- **QR code rotation:** When a code expires, the YouTube app seamlessly requests a new `device_code` and re-renders the QR — the user never needs to manually refresh the TV screen.
- **Token lifetimes:** `access_token` is short-lived (~1 hour); `refresh_token` is long-lived and allows the TV to silently re-authenticate without user interaction on subsequent sessions.

YouTube TV Sign-In: QR-Based Authentication (zAuth)

Source: <https://sharvilgupta.mintlify.app/introduction>

zAuth lets you sign in to YouTube on your TV instantly by scanning a QR code with your iPhone — no password typing required on your remote.

zAuth is YouTube's QR-code-based TV sign-in flow that eliminates the frustration of typing credentials with a TV remote. When you open YouTube on your TV and choose to sign in, your TV displays a QR code on screen. Simply point your iPhone camera at the code, tap the Google sign-in notification that appears, confirm your Google account, and you're instantly signed in — no passwords, no hunt-and-peck typing, no hassle.

How It Works

Dive into the technical details of the zAuth flow, from device code request to token exchange.

Step-by-Step Guide

Follow a guided walkthrough to sign in to YouTube on your TV using your iPhone.

Security & Privacy

Learn how zAuth keeps your Google credentials off your TV and your account secure.

Troubleshooting

Find answers to common issues with QR code scanning, expired codes, and account errors.

Sign In at a Glance

QR Code Appears on Your TV

Open YouTube on your TV and select **Sign In**. YouTube displays a unique QR code on your screen that is valid for 30 minutes.

Scan with Your iPhone Camera

Open the built-in Camera app on your iPhone (iOS 11 or later) and point it at the QR code on your TV. No third-party QR scanner app is needed.

Tap the Google Sign-In Notification

Your iPhone automatically detects the QR code and displays a banner notification. Tap it to open the Google sign-in page in your browser.

Confirm Your Account — You're Done

Select your Google account and tap **Allow** to grant YouTube access. Your TV signs in instantly and is ready to use.

zAuth is built on Google's **OAuth 2.0 Device Authorization Grant** ([RFC 8628](https://tools.ietf.org/html/rfc8628)) under the hood. Because the entire authentication flow happens on your iPhone's secure browser, your Google password is **never entered or stored on the TV**.

POST /o/device/code — Initiate Device Authorization

Source: <https://sharvilgupta.mintlify.app/reference/api/device-auth-endpoint>

Call POST /o/device/code to start the YouTube TV zAuth QR sign-in flow. Returns device_code, user_code, verification_uri_complete for QR rendering.

The device authorization endpoint initiates the zAuth flow. Your TV app calls this endpoint once to receive the device_code (used for polling), the user_code (the 8-character fallback code), and verification_uri_complete (the URL to encode as a QR code).

Endpoint

```
POST https://oauth2.googleapis.com/o/device/code
```

Request

The request body must be encoded as application/x-www-form-urlencoded.

Parameter (string)

Your OAuth 2.0 client ID from Google Cloud Console. Must be a **"TV and limited input device"** type client.

Parameter (string)

Space-separated list of OAuth scopes. For YouTube:

<https://www.googleapis.com/auth/youtube> openid profile

Example Request

```
curl -X POST https://oauth2.googleapis.com/o/device/code \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "client_id=YOUR_CLIENT_ID" \
-d "scope=https://www.googleapis.com/auth/youtube openid profile"
```

Response (200 OK)

device_code (string)

Opaque code your app stores and uses when polling the `/token` endpoint. Treat as a secret — do not display or log this value.

user_code (string)

Human-readable code (e.g., `GQVN-JKPZ`) displayed below the QR code for manual entry at `youtube.com/activate`.

verification_uri (string)

Base verification URL: `https://www.google.com/device`

verification_uri_complete (string)

Full URL including the `user_code` as a query parameter. **Encode this string as the QR code payload** so that scanning the QR pre-fills the code on the user's phone.

expires_in (integer)

Seconds until the `device_code` expires. Typically `1800` (30 minutes). After expiry, request a new device code and display a new QR code.

interval (integer)

Minimum number of seconds your app must wait between polling requests to `/token`. Typically `5`. Polling faster than this interval will return a `slow_down` error.

Example Response

```
{
  "device_code": "AH-1Ng0gTpDR9QKHN9tMHKGgV4R6x2ELMPj4b8_CqKk7szFb_mK2zHBQkCjfA",
  "user_code": "GQVN-JKPZ",
  "verification_uri": "https://www.google.com/device",
  "verification_uri_complete": "https://www.google.com/device?user_code=GQVN-JKPZ",
  "expires_in": 1800,
  "interval": 5
}
```

Error Responses

HTTP Status	Error	Description
400	<code>invalid_client</code>	<code>client_id</code> not found or not authorized for device flow
400	<code>invalid_scope</code>	Requested scope not allowed for this project
403	<code>access_denied</code>	API project quota exceeded

Device Authorization Error Codes — zAuth API Reference

Source: <https://sharvilgupta.mintlify.app/reference/api/error-codes>

Complete reference for all OAuth 2.0 and zAuth-specific error codes returned by Google's device authorization endpoints, with resolution steps.

Google's device authorization endpoints return structured error responses as JSON with an `error` field and an optional `error_description`. This reference documents every error code your YouTube TV app may encounter during the zAuth flow, along with the correct handling strategy for each.

Error Response Format

All error responses share the same JSON envelope:

```
{
  "error": "authorization_pending",
  "error_description": "Authorisation pending"
}
```

Device Authorization Errors

These errors are returned by the `/device/code` endpoint when initiating the zAuth flow.

invalid_client

The provided `client_id` is invalid or not authorized for the Device Authorization Grant.

Resolution:

- Verify your `client_id` in [Google Cloud Console](#) under **APIs & Services > Credentials**.
- Ensure the OAuth 2.0 client type is set to **"TV and Limited Input Device"**. Web application or desktop clients will be rejected.
- Confirm you are not accidentally using the `client_secret` in place of the `client_id`.

invalid_scope

One or more of the requested scopes are not permitted for this client or project.

Resolution:

- Check that all requested scopes are enabled for your Google Cloud project under **APIs & Services > Credentials**.
- Verify that the **YouTube Data API v3** is enabled under **APIs & Services > Enabled APIs**.
- Remove any scopes that your application does not genuinely require — Google enforces least-privilege scope usage.

access_denied — HTTP 403 (quota / API not enabled)

The project has exceeded its OAuth quota, or the required API is not enabled for this Google Cloud project.

This is a project-level denial returned by `/device/code` — distinct from the user-level `access_denied` returned during token polling when a user taps "Deny" on their phone (see [Token Polling Errors](#token-polling-errors) below).

Resolution:

- Enable the **YouTube Data API v3** in your Google Cloud project under **APIs & Services > Library**.
- If the API is already enabled, review your project's quota usage under **IAM & Admin > Quotas**.
- Request a quota increase via the Google Cloud Console if your application has high traffic volume.

Token Polling Errors

These errors are returned by the `/token` endpoint when using the `urn:ietf:params:oauth:grant-type:device_code` grant type during the polling phase.

authorization_pending — HTTP 400

The user has not yet confirmed the sign-in request on their phone or secondary device.

This is a **normal, expected response** during the polling loop — not a fatal error.

Resolution:

- Continue polling at the interval specified in the `/device/code` response (typically every 5 seconds).

- Do not increase the interval unless you also receive a `slow_down` error.
- Update any loading UI to indicate the app is waiting for user action.

slow_down — HTTP 400

Your app is polling the `/token` endpoint faster than the permitted interval.

Resolution:

- Immediately increase your polling interval by **5 seconds** from its current value (e.g., from 5s to 10s).
- Maintain the new increased interval for all subsequent requests — do not reset it back to the original.
- Continued `slow_down` responses will result in temporary suspension of the device code.

access_denied — HTTP 400

The user explicitly tapped "Deny" on their phone.

Resolution:

- Stop polling immediately — further polls will continue to return `access_denied`.
- Display a friendly error message to the user (e.g., "Sign-in was cancelled").
- Offer a clear call-to-action to restart the QR sign-in flow from the beginning.
- Do not automatically restart the flow — wait for explicit user intent.

expired_token — HTTP 400

The `device_code` TTL of 1800 seconds (30 minutes) has elapsed before the user authorized.

Resolution:

- Discard the expired `device_code` — it cannot be extended or reused.
- Make a new request to `/device/code` to obtain a fresh `device_code` and `user_code`.
- Display the new QR code and user code to the user.
- Consider proactively warning the user when the code is close to expiry (e.g., at the 5-minute mark).

invalid_grant — HTTP 400

The `device_code` has already been used (tokens were already issued) or the value is malformed.

Resolution:

- Do not reuse `device_code` values across multiple polling sessions.
- Once a successful `200` token response is received, stop polling and discard the `device_code`.
- If this error appears unexpectedly, treat it the same as `expired_token` — request a new device code.

Token Refresh Errors

These errors are returned by the `/token` endpoint when using the `refresh_token` grant type.

invalid_grant

The refresh token is invalid, expired, or has been revoked.

Common causes include:

- The user signed out of their Google account via myaccount.google.com.
- The user revoked your app's access under **Google Account > Security > Third-party apps**.
- Your Google Cloud project's OAuth consent screen was unpublished or deleted.
- A refresh token is only valid for a limited number of uses if token rotation is enabled.

Resolution:

- Discard the stored refresh token.

- Redirect the user through the **full QR sign-in flow** to re-authenticate and obtain new tokens.
- Communicate clearly to the user that they need to sign in again.

invalid_client

The ``client_id`` or ``client_secret`` provided in the refresh request is incorrect.

Resolution:

- Double-check the `client_id` and `client_secret` values against **Google Cloud Console > APIs & Services > Credentials**.
- Ensure you are not mixing up credentials from different OAuth clients or environments (dev vs. prod).

expired_token

The access token passed to a YouTube API call is expired.

Access tokens have a lifetime of approximately one hour (`expires_in: 3599`). A **401 Unauthorized** response from the YouTube API indicates the token has expired and must be refreshed.

Resolution:

- Use the stored `refresh_token` to request a new `access_token` from the `/token` endpoint.
- Implement automatic refresh logic in your API client layer so that token expiry is handled transparently.
- After a successful refresh, retry the original API request with the new access token.

HTTP Status Code Summary

Status Code	Used For	Meaning
200	<code>/token</code>	Success — tokens are present in the response body
400	<code>/token</code>	OAuth error — inspect the <code>error</code> field for the specific code
401	YouTube API calls	Access token is invalid or has expired
403	<code>/device/code</code>	Quota exceeded or required API is not enabled
400	<code>/token</code> polling	<code>authorization_pending</code> — user has not yet confirmed sign-in

Google also returns human-readable `error_description` strings in most error responses. While these strings are not suitable for display to end users, you should **log them during development** — they often contain specific details (e.g., which scope is invalid, or why a token was revoked) that make debugging significantly faster.

POST /token — Poll for Authorization and Refresh Tokens

Source: <https://sharvilgupta.mintlify.app/reference/api/token-endpoint>

Use POST `/token` to poll for device authorization during QR sign-in, and to silently exchange a refresh token for a new access token in the zAuth flow.

The token endpoint serves two distinct roles in the zAuth flow: (1) polling for user authorization during the QR sign-in phase, and (2) silently refreshing the access token after the initial sign-in. Both operations use the same endpoint with different `grant_type` values.

Endpoint

POST `https://oauth2.googleapis.com/token`

Operation 1: Poll for Device Authorization

After receiving a `device_code` from `/device/code` and displaying the QR code to the user, your app begins polling this endpoint at the cadence specified by `interval`. Polling continues until the user authorizes the request, denies it, or the `device_code` expires.

Request Parameters

Parameter (string)

Must be exactly `urn:ietf:params:oauth:grant-type:device_code`.

Parameter (string)

Your OAuth 2.0 client ID.

Parameter (string)

Your OAuth 2.0 client secret.

Parameter (string)

The `device_code` value returned by the `/device/code` response.

Example Request

```
curl -X POST https://oauth2.googleapis.com/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=urn:ietf:params:oauth:grant-type:device_code" \
-d "client_id=YOUR_CLIENT_ID" \
-d "client_secret=YOUR_CLIENT_SECRET" \
-d "device_code=AH-1Ng0gTpDR9QKH9tMHKGgV4R6x2ELMPj4b8_CqKk7szFb_mK2zHBQkCjFA"
```

Polling Response Codes

HTTP Status	error Value	Meaning
200	—	User authorized. Tokens are in the response body.
400	<code>authorization_pending</code>	User has not yet acted. Continue polling at the specified interval.
400	<code>slow_down</code>	Polling interval too fast. Increase interval by 5 seconds and maintain the new interval.
400	<code>access_denied</code>	User denied the request. Stop polling and offer to restart the flow.
400	<code>expired_token</code>	The <code>device_code</code> TTL elapsed. Request a new device code and display a new QR.

Success Response (200 OK)

```
{
  "access_token": "ya29.A0ARrdaM-Truncated_Example_Access_Token",
  "expires_in": 3599,
  "refresh_token": "1//0gTruncated_Example_Refresh_Token",
  "scope": "https://www.googleapis.com/auth/youtube openid profile",
  "token_type": "Bearer",
  "id_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.Truncated"
}
```

After sign-in, retrieve the user profile with GET <https://www.googleapis.com/oauth2/v3/userinfo> using the access token as a `Bearer` authorization header. See [Token Exchange](/reference/api/token-endpoint) for details.

Operation 2: Refresh Access Token

Access tokens expire after approximately one hour (`expires_in: 3599`). Use the `refresh_token` obtained during the initial sign-in to silently obtain a new access token without requiring the user to re-scan a QR code.

Request Parameters

Parameter (string)

Must be exactly `refresh_token`.

Parameter (string)

Your OAuth 2.0 client ID.

Parameter (string)

Your OAuth 2.0 client secret.

Parameter (string)

The refresh token stored securely from the initial sign-in response.

Example Request

```
curl -X POST https://oauth2.googleapis.com/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "grant_type=refresh_token" \
-d "client_id=YOUR_CLIENT_ID" \
-d "client_secret=YOUR_CLIENT_SECRET" \
-d "refresh_token=1//0gTruncated_Example_Refresh_Token"
```

Response

A successful refresh returns a new `access_token` and its expiry. Note that the `refresh_token` field is **not** returned in this response unless token rotation is explicitly enabled in your Google Cloud project.

```
{
  "access_token": "ya29.A0ARRdaM-New_Access_Token_Value",
  "expires_in": 3599,
  "scope": "https://www.googleapis.com/auth/youtube openid profile",
  "token_type": "Bearer"
}
```

The `client_secret` must **never** be embedded in client-side TV app code. If your secret is shipped inside the app binary, it can be extracted and abused. For production TV apps, proxy all token refresh requests through your own backend server, which holds the `client_secret` securely and returns only the resulting `access_token` to the device.

zAuth Authentication Flow Reference for YouTube TV

Source: <https://sharvilgupta.mintlify.app/reference/auth-flow>

Complete technical reference for the YouTube on TV zAuth device authorization flow, from device code request through token exchange.

This page documents the complete zAuth authentication flow as implemented in YouTube on TV. The flow follows RFC 8628 (OAuth 2.0 Device Authorization Grant) with Google-specific endpoint URLs and scope requirements.

Prerequisites

Google OAuth Client ID

A registered OAuth 2.0 client with `device` application type enabled in Google Cloud Console

YouTube Data API v3

API must be enabled in your Google Cloud project

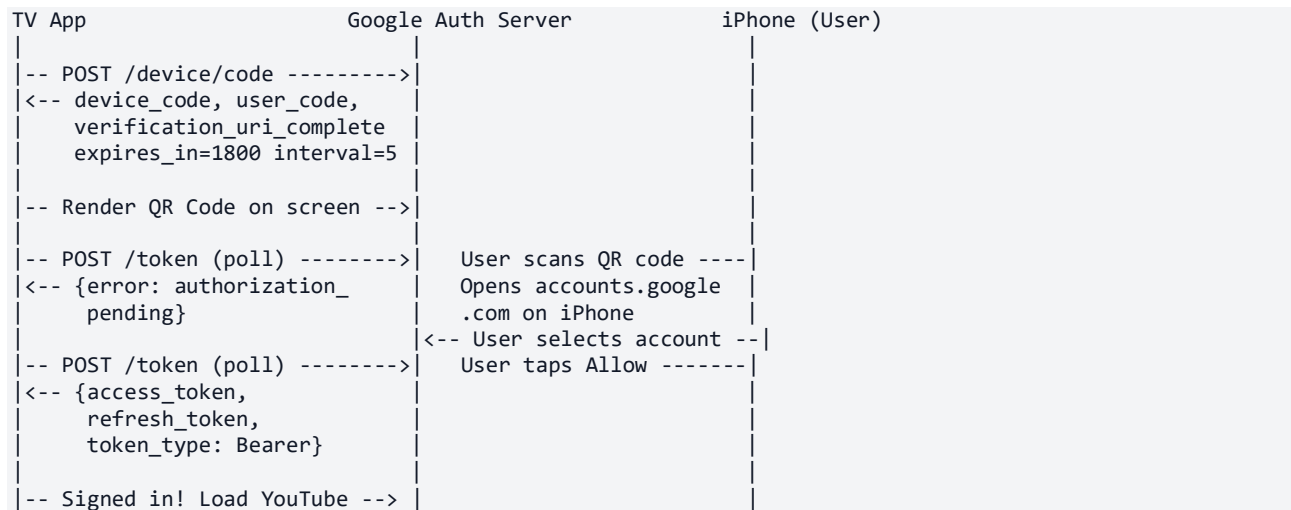
Allowed Scopes

Request only the YouTube scopes your app needs

HTTPS Connectivity

TV device must reach `oauth2.googleapis.com`

Complete Flow Diagram



Phase 1: Device Code Request

The TV app initiates the flow by posting to Google's device authorization endpoint. The only required parameters are your OAuth client ID and the scopes your app needs.

```
POST https://oauth2.googleapis.com/device/code
Content-Type: application/x-www-form-urlencoded
client_id=YOUR_CLIENT_ID&scope=https%3A%2F%2Fwww.googleapis.com%2Fauth%2Fyoutube
```

Google responds with all the data you need to display a QR code and begin polling:

```
{
  "device_code": "4/4-GMMhmHCXhWEzkobqIHGG_EnNYysAkukHspeYUk9E8",
  "user_code": "GQVN-JKPZ",
  "verification_uri": "https://www.google.com/device",
  "verification_uri_complete": "https://www.google.com/device?user_code=GQVN-JKPZ",
  "expires_in": 1800,
  "interval": 5
}
```

Phase 2: QR Code Rendering

The `verification_uri_complete` field is the exact string to encode as a QR code — it embeds the `user_code` in the URL so the user doesn't need to type anything manually. Display the QR code on the TV screen immediately after receiving this response.

```
import qrcode
uri = device_code_response["verification_uri_complete"]
qr = qrcode.make(uri)
qr.show() # Display on TV screen
```

Also display the `user_code` as visible text below the QR code as a manual entry fallback for users who cannot scan. See [QR Code Generation](/reference/qr-code-generation) for full rendering requirements and platform-specific code examples.

Phase 3: Polling

While the QR code is displayed, your app must poll Google's token endpoint at the specified interval to detect when the user approves the sign-in. See the [Polling Mechanism](/reference/polling-mechanism) page for the full polling implementation, error handling, and backoff strategy.

Phase 4: Token Exchange

Once the user confirms sign-in on their iPhone, the next poll returns an access token and refresh token. See the [Token Exchange](/reference/token-exchange) page for the full token response structure, secure storage guidance, and how to silently refresh the session.

Polling for zAuth Authorization Status on YouTube TV

Source: <https://sharvilgupta.mintlify.app/reference/polling-mechanism>

How to poll Google's token endpoint during the YouTube TV zAuth flow, handle all four error states including `slow_down`, and implement correct backoff.

While the QR code is displayed on the TV, your app must poll Google's token endpoint at regular intervals to check whether the user has confirmed the sign-in on their iPhone. This page covers correct polling behavior, error handling, and backoff strategies.

The Polling Request

Send a `POST` to Google's token endpoint with your client credentials, the `device_code` from the authorization response, and the device code grant type:

```
POST https://oauth2.googleapis.com/token
Content-Type: application/x-www-form-urlencoded
client_id=YOUR_CLIENT_ID
&client_secret=YOUR_CLIENT_SECRET
&device_code=4/4-GMMhmHCXhWEzkobqIHGG_EnNYYSAkukHspeYUk9E8
&grant_type=urn:ietf:params:oauth:grant-type:device_code
```

Polling Response States

Each poll returns one of the following outcomes. Your implementation must handle all of them:

HTTP Status	error field	Meaning	Action
200 OK	— (has <code>access_token</code>)	User confirmed	Extract tokens, sign in
400	<code>authorization_pending</code>	User hasn't confirmed yet	Wait <code>interval</code> seconds, poll again
400	<code>slow_down</code>	Polling too fast	Increase interval by 5s, then poll
400	<code>access_denied</code>	User tapped Deny	Abort, show error, offer retry
400	<code>expired_token</code>	<code>device_code</code> TTL elapsed	Request new device code, show new QR

Reference Implementation

```
import time
import requests
def poll_for_authorization(client_id, client_secret, device_code, interval):
    poll_interval = interval # Start with server-specified interval
    while True:
        time.sleep(poll_interval)
        response = requests.post(
            'https://oauth2.googleapis.com/token',
            data={
```

```

'client_id': client_id,
'client_secret': client_secret,
'device_code': device_code,
'grant_type': 'urn:ietf:params:oauth:grant-type:device_code',
}
)
data = response.json()
if response.status_code == 200 and 'access_token' in data:
    return data # Success!
error = data.get('error')
if error == 'authorization_pending':
    continue # Normal – user hasn't confirmed yet
elif error == 'slow_down':
    poll_interval += 5 # Back off by 5 seconds
elif error == 'access_denied':
    raise Exception('User denied access')
elif error == 'expired_token':
    raise Exception('Device code expired – request a new one')
else:
    raise Exception(f'Unexpected error: {error}')
async function pollForAuthorization(clientId, clientSecret, deviceCode, interval) {
    let pollInterval = interval;
    while (true) {
        await new Promise(resolve => setTimeout(resolve, pollInterval * 1000));
        const response = await fetch('https://oauth2.googleapis.com/token', {
            method: 'POST',
            headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
            body: new URLSearchParams({
                client_id: clientId,
                client_secret: clientSecret,
                device_code: deviceCode,
                grant_type: 'urn:ietf:params:oauth:grant-type:device_code',
            }),
        });
        const data = await response.json();
        if (response.ok && data.access_token) return data; // Success
        switch (data.error) {
            case 'authorization_pending': break; // Keep polling
            case 'slow_down': pollInterval += 5; break; // Back off
            case 'access_denied': throw new Error('User denied access');
            case 'expired_token': throw new Error('Device code expired');
            default: throw new Error(`Unexpected error: ${data.error}`);
        }
    }
}

```

Polling Rules

Violating Google's polling rules can result in your OAuth client being rate-limited or suspended:

- **Never poll faster** than the **interval** value returned in the device code response
- **If you receive `slow\_down`**, add 5 seconds to your current interval and keep that higher interval for the remainder of the session — do not reset it
- **Polling too aggressively** will result in your client being rate-limited across all users
- **Always stop polling immediately** on **access_denied** or **expired_token** — continued polling after these errors is wasted traffic and may trigger abuse detection

Minimum Interval

Google sets **interval** to **5 seconds** by default. Never hardcode a faster polling interval — always read and respect the **interval** value from the device code response. If Google adjusts this value in the future, your implementation will adapt automatically.

Generating the zAuth QR Code for YouTube TV Sign-In

Source: <https://sharvilgupta.mintlify.app/reference/qr-code-generation>

How to generate, render, and auto-rotate the OAuth 2.0 device authorization QR code for YouTube TV sign-in screens using `verification_uri_complete`.

After receiving the `verification_uri_complete` from Google's device authorization endpoint, your TV app must render it as a scannable QR code on the TV screen. This page covers QR generation, display requirements, and automatic rotation on expiry.

The QR Payload

The `verification_uri_complete` is a URL of the form:

```
https://www.google.com/device?user_code=GQVN-JKPZ
```

This is the **exact string** to encode into the QR code — no modification, shortening, or re-encoding is needed.

The `user_code` is already embedded in the URL so the user is taken directly to the confirmation screen after scanning, with no manual code entry required.

QR Generation Code Examples

```
import qrcode
from PIL import Image
def generate_youtube_qr(verification_uri_complete: str) -> Image:
    qr = qrcode.QRCode(
        version=None, # auto-size
        error_correction=qrcode.constants.ERROR_CORRECT_M,
        box_size=10,
        border=4,
    )
    qr.add_data(verification_uri_complete)
    qr.make(fit=True)
    return qr.make_image(fill_color="black", back_color="white")
const QRCode = require('qrcode');
async function generateYouTubeQR(verificationUriComplete) {
    const opts = {
        errorCorrectionLevel: 'M',
        type: 'image/png',
        margin: 4,
        color: { dark: '#000000', light: '#FFFFFF' },
    };
    const dataUrl = await QRCode.toDataURL(verificationUriComplete, opts);
    return dataUrl; // Use as <img src> on TV UI
}
import CoreImage
func generateQRCode(from string: String) -> UIImage? {
    let data = string.data(using: .ascii)
    guard let filter = CIFilter(name: "CIQRCodeGenerator") else { return nil }
    filter.setValue(data, forKey: "inputMessage")
    filter.setValue("M", forKey: "inputCorrectionLevel") // Medium error correction
    guard let ciImage = filter.outputImage else { return nil }
    let scale = 10.0
    let transform = CGAffineTransform(scaleX: scale, y: scale)
    let scaledImage = ciImage.transformed(by: transform)
    return UIImage(ciImage: scaledImage)
}
import com.google.zxing.BarcodeFormat
import com.google.zxing.EncodeHintType
import com.google.zxing.qrcode.QRCodeWriter
import android.graphics.Bitmap
fun generateYouTubeQR(verificationUriComplete: String, size: Int = 512): Bitmap {
    val hints = mapOf(
        EncodeHintType.ERROR_CORRECTION to "M",
        EncodeHintType.MARGIN to 4
    )
}
```

```

)
val writer = QRCodeWriter()
val bitMatrix = writer.encode(
    verificationUriComplete,
    BarcodeFormat.QR_CODE,
    size,
    size,
    hints
)
val bitmap = Bitmap.createBitmap(size, size, Bitmap.Config.RGB_565)
for (x in 0 until size) {
    for (y in 0 until size) {
        bitmap.setPixel(
            x, y,
            if (bitMatrix[x, y]) android.graphics.Color.BLACK
            else android.graphics.Color.WHITE
        )
    }
}
return bitmap
}

```

Display Requirements

When rendering the QR code on the TV screen, follow these requirements to ensure reliable scanning:

- **Minimum rendered size:** 200×200 pixels — larger is better on high-resolution TV displays
- **High contrast:** Black modules on a white background; do not invert or apply color themes
- **Quiet zone:** Maintain a 4-module white border around all edges of the QR code
- **Manual fallback:** Display the `user_code` as visible text below the QR code for users who prefer to type it at google.com/device
- **TTL countdown:** Show a countdown timer based on `expires_in` so users know how long the code remains valid

QR Code Expiry & Auto-Rotation

The `expires_in` field (1800 seconds = 30 minutes) defines the lifetime of the device code and its associated QR code. Once elapsed, the `device_code` is no longer valid and polling will return an `expired_token` error.

Your app must detect expiry and automatically request a new device code, then re-render the QR code seamlessly:

```

import time
import asyncio
async def manage_qr_lifecycle(client_id, scope):
    while True:
        response = request_device_code(client_id, scope)
        render_qr(response['verification_uri_complete'])
        expiry = time.time() + response['expires_in']
        while time.time() < expiry:
            result = poll_for_token(response['device_code'])
            if result.get('access_token'):
                return result # Sign-in complete
            await asyncio.sleep(response['interval'])
        # Code expired – loop and request a new one
        show_message("Code expired, refreshing...")

```

Error Correction Level

Use **M` (Medium)** error correction — it provides 15% data recovery capacity and produces a less dense QR code than **Q** or **H**. TV displays can introduce slight pixel distortion at viewing angles, and a less dense code is easier for iPhone cameras to decode reliably from across a room.

zAuth Token Exchange and Session Management Reference

Source: <https://sharvilgupta.mintlify.app/reference/token-exchange>

How to handle the zAuth token response, retrieve the signed-in Google user profile, store tokens securely, and silently refresh YouTube TV sessions.

When the user confirms sign-in on their iPhone, the next poll to Google's token endpoint returns an access token and refresh token. This page covers the successful token response structure, secure storage guidance, and how to use the refresh token to maintain the YouTube TV session indefinitely.

Successful Token Response

A **200 OK** response from the polling endpoint signals that the user has approved the sign-in. The body contains all the tokens your app needs:

```
{
  "access_token": "ya29.A0ARrdAM...",
  "expires_in": 3599,
  "refresh_token": "1//04dEXdGp...",
  "scope": "https://www.googleapis.com/auth/youtube openid",
  "token_type": "Bearer",
  "id_token": "eyJhbGciOiJIUzI1NiJ9..."
}
```

Field	Type	Description
access_token	string	Bearer token for YouTube API calls (valid ~1 hour)
expires_in	integer	Seconds until the access token expires (typically 3599)
refresh_token	string	Long-lived token to obtain new access tokens (valid until revoked)
scope	string	Space-separated list of granted OAuth scopes
token_type	string	Always "Bearer"
id_token	string	JWT containing the user's Google identity claims

Decoding the id_token

The **id_token** is a JSON Web Token (JWT) issued by Google. It contains the signed-in user's identity claims — useful for displaying their name, email, and profile photo without making an additional API call.

```
import jwt # PyJWT
# Decode without verification for display purposes only
# Always verify the signature in production
claims = jwt.decode(
  id_token,
  options={"verify_signature": False}
)
print(claims['email']) # user@gmail.com
print(claims['name']) # Display name
print(claims['picture']) # Profile photo URL
print(claims['sub']) # Stable Google user ID
```

In production, always verify the **id_token** signature against Google's public keys before trusting its claims. Use the ``google-auth`` library (Python) or an equivalent for your platform — it handles key fetching, signature verification, and expiry checking automatically.

Retrieving the Signed-In User Profile

In addition to decoding the `id_token` locally, you can fetch the signed-in Google account's profile directly from Google's `UserInfo` endpoint using the `access_token`. This is useful when you need a canonical, server-verified view of the user's identity.

```
GET https://www.googleapis.com/oauth2/v3/userinfo
Authorization: Bearer ya29.A0ARRdaM...
```

A successful response returns a JSON object with the user's Google account details:

```
{
  "sub": "110048951234567890123",
  "name": "Jane Smith",
  "given_name": "Jane",
  "family_name": "Smith",
  "picture": "https://lh3.googleusercontent.com/a/...",
  "email": "jane.smith@gmail.com",
  "email_verified": true
}
```

Field	Type	Description
<code>sub</code>	string	Stable, unique Google user ID — use this as your primary user identifier
<code>name</code>	string	User's full display name
<code>given_name</code>	string	First name
<code>family_name</code>	string	Last name
<code>picture</code>	string	URL of the user's Google profile photo
<code>email</code>	string	User's Google account email address
<code>email_verified</code>	boolean	Whether Google has verified the email address

The `sub` claim is the most stable identifier — it never changes even if the user updates their email address. Use `sub` (not `email`) as your primary key when storing user records.

Making Authenticated YouTube API Calls

Use the `access_token` as a `Bearer` token in the `Authorization` header of every YouTube Data API v3 request:

```
GET https://www.googleapis.com/youtube/v3/channels?part=snippet&mine=true
Authorization: Bearer ya29.A0ARRdaM...
```

The access token expires after approximately one hour (`expires_in` seconds). When a YouTube API call returns a `401 Unauthorized` response, use the refresh token to obtain a new access token silently.

Refreshing the Access Token

Send the stored `refresh_token` to Google's token endpoint to obtain a new `access_token` without requiring the user to re-authenticate:

```
POST https://oauth2.googleapis.com/token
Content-Type: application/x-www-form-urlencoded
client_id=YOUR_CLIENT_ID
&client_secret=YOUR_CLIENT_SECRET
&refresh_token=1//04dEXdGp...
&grant_type=refresh_token
```

The response has the same structure as the original token response, but **does not include a new** `refresh_token` unless refresh token rotation is explicitly enabled for your OAuth client. Reuse the original refresh token for subsequent refreshes.

```
{
  "access_token": "ya29.A0ARRdaM_NEW...",
  "expires_in": 3599,
  "scope": "https://www.googleapis.com/auth/youtube openid",
  "token_type": "Bearer"
}
```

Token Storage Guidance

Tokens are credentials — treat them with the same care as passwords:

- **Store the `refresh\_token`** in encrypted persistent storage on the TV device (Android Keystore, tvOS Keychain, iOS Keychain)
- **Never log or transmit tokens** in plaintext — exclude them from analytics events, crash reports, and debug logs
- **The `access\_token` can be kept in memory only** — it is short-lived and does not need to survive app restarts
- **Clear all tokens on user sign-out** — both from memory and from persistent storage — and revoke the refresh token via <https://oauth2.googleapis.com/revoke> if possible
- **Use platform secure storage** appropriate for your target: Android Keystore for Android TV, Keychain Services for Apple TV, and an equivalent encrypted credential store for other platforms

Privacy and Data Access for YouTube on TV QR Sign-In

Source: <https://sharvilgupta.mintlify.app/security/data-privacy>

What data YouTube accesses when you sign in via QR code on your TV, and how Google protects your personal information during zAuth.

When you sign in to YouTube on your TV using the zAuth QR flow, you grant YouTube access to specific data in your Google account. This page explains exactly what data is shared, what is not, and how you can control your privacy settings.

Data Accessed by YouTube on TV

YouTube on TV requests the minimum set of permissions necessary to deliver a personalized playback experience. The table below describes each type of data accessed, why it is needed, and what controls are available to you.

Data Type	Why It's Needed	How to Limit It
Google Account Identity	Verify who is signing in	Cannot be limited (required to sign in)
YouTube Watch History	Personalize recommendations	Pause at myaccount.google.com/data-and-privacy
YouTube Subscriptions	Show your subscribed channels	Manage in YouTube settings
Profile Name & Photo	Display in YouTube TV header	Edit at myaccount.google.com

What Is NOT Accessed

YouTube on TV requests only YouTube-scoped permissions. The following Google services and data types are **not** accessible to YouTube on TV and are never part of the zAuth authorization request:

- Gmail
- Google Drive
- Google Photos
- Google Pay
- Contacts
- Location data
- Calendar
- Google Search history

Data Stored on the TV

After sign-in, a small set of session data is retained locally on the TV device to support ongoing playback and the account switcher UI. No passwords or sensitive Google account data are ever stored on the TV.

- **Access token** — ephemeral, valid for approximately 1 hour, then replaced automatically
- **Refresh token** — stored in the app's encrypted on-device storage; used only to silently renew the access token
- **Google display name and profile photo** — cached locally to populate the account switcher UI in YouTube on TV

Google's Privacy Controls

You can review and manage the data YouTube on TV accesses at any time through your Google Account settings.

Review Activity

Manage your YouTube watch history and activity, pause history tracking, or delete past activity entirely.

Manage Connected Apps

Review all third-party apps with access to your Google account and revoke YouTube TV's access at any time.

Children's Accounts

Google Family Link managed accounts (children under 13) may have restrictions that prevent signing in to YouTube on TV. If a child's account is managed by Family Link, a parent or guardian must approve the device connection before the zAuth flow can complete. Parents can manage device permissions from the Family Link app or at families.google.com.

YouTube TV Token Lifecycle, Rotation, and Revocation

Source: <https://sharvilgupta.mintlify.app/security/token-lifecycle>

Understand how YouTube on TV manages access tokens and refresh tokens for your TV session, including expiry, silent rotation, and revocation.

When you complete the zAuth QR sign-in, YouTube on TV receives two tokens from Google: a short-lived access token and a long-lived refresh token. Understanding how these tokens work helps you manage your TV sessions and know when you might be asked to sign in again.

Token Types

YouTube on TV uses two distinct token types throughout the lifetime of your session. Each serves a different purpose and operates on a different timescale.

Token	Lifetime	Purpose
Access Token	1 hour	Authorizes YouTube API calls
Refresh Token	Until revoked	Obtains new access tokens silently

How Token Refresh Works

After the initial sign-in, the TV manages your session entirely in the background. You will not be prompted to scan a QR code again unless something explicitly invalidates your session.

TV uses the access token

YouTube on TV includes the access token in all YouTube API requests — for example, fetching your subscriptions feed, resuming playback, or saving watch progress.

Access token expires

After approximately 1 hour, the access token expires. Any API call made with an expired token returns a **401 Unauthorized** response.

TV sends the refresh token to Google

The YouTube TV app automatically detects the expired token and sends the refresh token to Google's token endpoint (<https://oauth2.googleapis.com/token>) to request a replacement.

Google issues a new access token

Google validates the refresh token and returns a fresh access token. In some cases, Google also rotates the refresh token itself, replacing the old one with a new long-lived token.

Playback continues uninterrupted

The TV resumes its API calls with the new access token. This entire exchange happens silently in the background — you never see a sign-in prompt or any interruption to your viewing experience.

When You'll Be Asked to Sign In Again

In most cases, your TV stays signed in indefinitely. However, the following events will invalidate your session and require a new zAuth QR sign-in:

- You manually sign out from YouTube on TV
- You revoke access from myaccount.google.com
- Your Google account password is changed (Google invalidates all tokens associated with the account)
- Your Google account has 2-Step Verification and a new trusted device policy is enforced
- The TV app data is cleared or the app is reinstalled

Revoking TV Access

You can remove a TV's access to your Google account at any time from any device. Revocation is immediate — the TV cannot acquire a new access token after you complete these steps.

Open your Google Account security page

Go to myaccount.google.com/security on any device — your phone, computer, or tablet.

Find Third-party app access

Scroll down to the section labelled **"Third-party apps with account access"** and click to expand it.

Locate YouTube on TV

Find **"YouTube on TV"** in the list (individual devices may appear under their own name if you have multiple TVs signed in).

Remove access

Click **"Remove Access"** next to the entry you want to revoke.

Sign-out takes effect

The TV is immediately signed out on its next YouTube API call. Any content currently playing may finish buffering, but no new requests to your account will be authorized.

Token Security

Refresh tokens are stored securely in the TV app's encrypted on-device storage and are never exposed through the TV's user interface. **Never share screenshots of your QR code** — a valid, unexpired QR code contains a live `user_code` that would allow anyone who scans it to authorize their own session on your Google account before you do.

How zAuth Keeps Your YouTube on TV QR Sign-In Secure

Source: <https://sharvilgupta.mintlify.app/security/zauth-protocol>

zAuth uses OAuth 2.0 Device Authorization Grant (RFC 8628) to authenticate TV sign-ins securely without exposing your Google password on the TV.

zAuth is YouTube's implementation of the OAuth 2.0 Device Authorization Grant (RFC 8628), a standard specifically designed for devices with limited input capabilities — like smart TVs. This protocol ensures your Google credentials are only ever entered on a trusted device (your iPhone), never on the TV.

Why TVs Can't Handle Passwords Directly

Typing a password using a TV remote is tedious and error-prone — remote control keyboards require navigating an on-screen grid one character at a time. Beyond the inconvenience, TVs in shared spaces (living rooms, hotel rooms, waiting areas) present a real screen-sniffing risk: anyone in the room could observe your password being entered character by character. zAuth eliminates both problems by moving credential entry entirely off the TV and onto your iPhone, where input is private and familiar.

The zAuth Security Model

Credentials Never Leave Your Phone

Your Google password is entered only on your iPhone. At no point does the TV receive, transmit, or store your password — it only ever sees an authorization token issued after you've already authenticated.

Short-Lived Codes

Device codes and the QR codes that encode them expire after 30 minutes. If you don't complete sign-in within that window, the code is invalidated and a new one must be generated.

One-Time Authorization

Each QR code is single-use. Once you scan it and confirm your Google account, the underlying `user_code` is immediately invalidated on Google's servers and cannot be reused by anyone else.

Revocable Tokens

You can revoke a TV's access to your Google account at any time from myaccount.google.com. Revocation takes effect immediately on the next API call the TV makes.

OAuth Scopes Requested

When you authorize YouTube on your TV, zAuth requests only the permissions needed for YouTube playback and personalization. No other Google services are granted access.

Scope	Purpose
https://www.googleapis.com/auth/youtube	Access your YouTube account
https://www.googleapis.com/auth/youtube.readonly	Read watch history and subscriptions
<code>openid</code>	Verify your identity
<code>profile</code>	Read your display name and profile photo

How the QR Code Is Protected

The QR code encodes a `verification_uri_complete` URL that contains a one-time `user_code` generated by Google's authorization server. This code is single-use, time-limited to 30 minutes, and server-side invalidated the moment it is used or expires — whichever comes first. Even if someone photographs your TV screen, a used or expired QR code is cryptographically worthless.

Man-in-the-Middle Protection

All communication in the zAuth flow — between the TV and Google's authorization server, and between your iPhone and Google during QR scan — is protected by HTTPS/TLS. When you scan the QR code, your iPhone opens `accounts.google.com` directly. No intermediate server, proxy, or YouTube-owned relay sits between your phone and Google's authentication endpoint, which means there is no point in the flow where a third party could intercept or replay your credentials.

Sign-In Complete: You Are Now Signed in to YouTube on TV

Source: <https://sharvilgupta.mintlify.app/sign-in/completed>

Your Google account is now linked to YouTube on your TV. Learn what happens next, how long your session lasts, and how to sign out when needed.

Once you tap **Allow** on your iPhone, the YouTube app on your TV automatically detects the confirmed authorization and signs you in — no further action needed on the TV remote. Your personalized YouTube experience loads immediately.

What Happens Automatically

As soon as the authorization is confirmed, YouTube on your TV takes care of the rest:

- **Session activated within 5 seconds** — the TV polls Google's authorization server and detects your approval almost instantly.
- **Your YouTube homepage loads** — personalized with your subscriptions, Recommended videos, and continue-watching queue.
- **Watch history begins syncing** — videos you watch on your TV are added to your Google account watch history in real time.
- **Subscriptions are recognized** — any active **YouTube Premium** or **YouTube TV** subscription tied to your Google account is detected and applied automatically; no separate activation is needed on the TV.

How Long Does the Session Last?

The YouTube TV session is powered by a **long-lived refresh token** issued by Google at the moment you tapped Allow. This token remains valid until it is explicitly revoked, meaning:

- You **stay signed in** across TV restarts, app updates, and complete power cycles.
- There is no automatic session timeout — you won't be asked to re-authenticate on a schedule.

You will only be prompted to sign in again if:

1. You **sign out manually** from within the YouTube app on your TV.
2. You **revoke access** to YouTube on your TV from your Google account settings at `myaccount.google.com`.
3. The **YouTube app is reset** or your TV is factory reset, which clears stored tokens.

Signing Out

Open YouTube on your TV

Using your remote, navigate to and open the **YouTube** app.

Go to your Account

Select the **Account** icon in the top-right corner of the YouTube interface to open the account menu.

Select Sign Out

Scroll down and select **Sign Out**, then confirm when prompted. YouTube will return to the signed-out home screen and display a new QR code if you wish to sign in again.

Managing Your Connected Devices

You can review and revoke YouTube TV access at any time from your Google Account security settings — no need to have the TV nearby. Visit myaccount.google.com/security on any device, scroll to the "Third-party apps with account access" section, and locate YouTube. From there you can tap **Remove access** to immediately invalidate the TV session's refresh token. The TV will be signed out the next time it attempts to refresh its authorization.

Next Steps

Troubleshooting

Having issues with the sign-in flow? Browse common questions and fixes in the FAQ.

Security & Privacy

Learn how OAuth 2.0 Device Authorization Grant keeps your credentials secure and what access YouTube on your TV is granted.

Step 3: Confirm Your Google Account on Your iPhone

Source: <https://sharvilgupta.mintlify.app/sign-in/confirm-account>

Choose which Google account to link to YouTube on your TV and tap Allow to complete the sign-in confirmation on your iPhone safely and securely.

After scanning the QR code, Google opens on your iPhone and asks which account you want to sign in with on your TV. This step keeps your credentials secure — you authenticate on your phone, never on the TV itself.

Account Selection Screen

When Google loads on your iPhone, you'll see:

- A **Google "Sign in" screen** listing all Google accounts currently signed in on your iPhone, each shown with its profile photo and email address.
- A brief **permissions description** outlining what YouTube will have access to once you confirm (e.g., "YouTube will be able to access your YouTube account, subscriptions, and basic profile information").
- An **Allow** and **Deny** button pair at the bottom of the screen for you to confirm or cancel the sign-in request.

Choosing Your Account

Select your Google account

Review the list of Google accounts shown on screen. Tap the **account** you want to use for YouTube on your TV. If you only have one account signed in, it will be pre-selected.

Review the permissions screen

Google displays the permissions YouTube is requesting — this typically includes access to your **YouTube account** (watch history, subscriptions, recommendations) and your **basic profile info** (name and profile photo). Read through these to confirm you're comfortable with the access being granted.

Tap Allow

Tap the **Allow** button to confirm the sign-in. This securely sends the authorization back to YouTube on your TV.

Confirmation and close

Your iPhone displays a confirmation message: "**You are now signed in on your TV.**" You can safely close the browser tab — no further action is required on your iPhone.

Signing in with a Different Account

If the Google account you want to use isn't listed, you'll need to add it to your iPhone first. Open **Safari** and navigate to **accounts.google.com**, then sign in with the account you'd like to use. Once added, return to the YouTube sign-in flow: scan the QR code again (or re-enter the activation code at youtube.com/activate) and the new account will appear in the selection list.

Denying Access

Tapping **Deny** on the permissions screen cancels the sign-in flow entirely. Your TV will display an authorization error and return to the sign-in screen. No account will be linked and no data is shared. You can restart the sign-in process at any time by navigating back to the QR code screen on your TV.

Multiple Google Accounts

If you have several Google accounts on your iPhone — for example, a personal account and a work account — you can choose exactly which one to link to YouTube on your TV. Simply tap the desired account on the selection screen. The YouTube TV session will be tied exclusively to whichever account you chose; recommendations, subscriptions, watch history, and any YouTube Premium or YouTube TV subscription will reflect that account. You can always sign out from the TV and repeat the process to switch to a different account.

Step 1: Display the YouTube TV QR Code on Your Screen

Source: <https://sharvilgupta.mintlify.app/sign-in/display-qr-code>

Learn how to reach the YouTube TV sign-in screen on any supported TV platform and display the QR code ready to scan with your iPhone.

The first step in the YouTube TV sign-in process is reaching the QR code screen on your TV. This page shows you exactly how to get there on any supported TV platform.

Make sure your iPhone and TV are in the same room — you just need to be close enough to point your camera at the screen. No Wi-Fi pairing required.

How to Get to the Sign-In Screen

Android TV / Google TV

Open YouTube

From the home screen, navigate to the YouTube app and press **Select** to open it.

Go to Sign In

Using your remote, navigate to the top-right of the screen and select the **Sign In** button.

View the QR Code

YouTube displays the sign-in QR code on screen. Select **Sign in with your phone or computer** if prompted to reveal the QR code.

Apple TV

Open YouTube

From the Apple TV home screen, select the YouTube app. If you haven't installed it, find it in the App Store first.

Go to Sign In

Navigate to the **Account** tab in the YouTube app and select **Sign In**.

View the QR Code

The QR code screen appears automatically. If you see a text code instead, select **Show QR Code** to switch views.

Amazon Fire TV

Open YouTube

From the Fire TV home screen, locate the YouTube app in your apps list or search for it, then press **Select** to open it.

Go to Sign In

Navigate to the left-hand menu, scroll down to **Sign In**, and press **Select**.

View the QR Code

YouTube presents the sign-in QR code. If you land on the text-code screen, choose **Use QR Code** to switch.

Samsung / LG Smart TV

Open YouTube

Press the **Home** button on your remote, navigate to the YouTube app, and press **Enter** or **OK** to launch it.

Go to Sign In

Select the **Account** icon (top-right corner) and choose **Sign In** from the menu.

View the QR Code

The QR code is displayed on screen. On some models you may need to select **Sign in with mobile** to reach the QR code view.

Chromecast

Open YouTube

From the Google TV interface on your Chromecast, navigate to the YouTube app and press **Select** to open it.

Go to Sign In

Open the YouTube menu, scroll to the **Sign In** option, and press **Select**.

View the QR Code

YouTube displays the sign-in QR code alongside the device activation code. You're ready to scan.

What the QR Code Screen Shows

Once you've navigated to the sign-in screen, you'll see the following on your TV:

- **A large, scannable QR code** — the primary way to authenticate using your iPhone camera.
- **An 8-character alphanumeric code** (e.g., **WXYZ-1234**) — displayed below the QR code for manual entry as a fallback option.
- **A URL** — youtube.com/activate where you can type the code manually from any browser.

- **An expiry timer** — a countdown showing how long the current QR code remains valid (30 minutes).

QR Code Expiry

The QR code expires after **30 minutes**. If it expires before you scan it, YouTube will automatically generate a new one — just wait a moment and a fresh code will appear on screen. You do not need to navigate back to this screen or restart the app.

Step 2: Scan the YouTube TV QR Code with Your iPhone

Source: <https://sharvilgupta.mintlify.app/sign-in/scan-with-iphone>

Use your iPhone Camera app to scan the YouTube TV QR code in seconds. No third-party app needed — iOS 11 and later scan QR codes natively.

Your iPhone's built-in Camera app can scan the YouTube TV QR code directly — you don't need a separate QR scanner app. This works on any iPhone running iOS 11 or later.

Scanning with the Camera App

Open the native Camera app

Open the **Camera** app that came with your iPhone. Do not use a third-party QR scanner app — the native Camera app provides a direct, trusted link to Google's authentication page.

Point the camera at the QR code

Hold your iPhone up and aim the camera at the QR code displayed on your TV screen. Keep it steady for **1–2 seconds** — you don't need to tap anything. The QR code should fill the center of your viewfinder; a distance of 20–40 cm from the screen works well.

Tap the notification banner

A notification banner appears at the top of your iPhone screen reading "**Open 'accounts.google.com'**" — tap it to proceed. This banner disappears after a few seconds, so tap it promptly.

Complete sign-in in Safari

Safari (or your default browser) opens automatically, loading the Google sign-in page with the device code **pre-filled**. You'll be taken straight to the account selection step — no copying or typing required.

Troubleshooting Scan Issues

Camera doesn't detect the QR code

If your camera hovers over the QR code without recognizing it, try the following:

- **Improve lighting** — make sure the TV screen is bright and the room isn't too dark or has direct glare hitting the screen.
- **Adjust your distance** — hold your phone **20–40 cm** away from the screen. Being too close or too far prevents detection.
- **Clean your camera lens** — a smudged lens can interfere with recognition. Gently wipe it with a soft cloth.
- **Reduce screen glare** — tilt the TV slightly or move to a different angle to minimize reflections on the screen.
- **Increase TV brightness** — temporarily boost your TV's brightness setting to make the QR code more defined.

No notification appears after scanning

If the camera seems to detect the code (you see the focus square lock on) but no banner appears:

1. Check that QR code scanning is enabled on your iPhone: go to **Settings → Camera** and make sure the **Scan QR Codes** toggle is turned **on**.
2. Ensure you're using the rear camera, not the front-facing (selfie) camera.
3. Dismiss any other notifications that may be blocking the banner and try scanning again.

Camera app won't open

If tapping the Camera app icon does nothing or it crashes immediately:

- **Use Control Center** — swipe down from the top-right corner of your screen (or up from the bottom on older iPhones) and tap the camera icon to open it directly.
- **Lock and unlock your iPhone** — press the side button to lock the screen, then unlock it and try opening Camera again.
- **Restart your iPhone** — if the camera is unresponsive, a quick restart usually resolves temporary app freezes.

Alternative: Manual Code Entry

If scanning isn't working, you can sign in using the **8-character text code** shown on your TV screen instead. On your iPhone, open Safari and navigate to **youtube.com/activate**. When prompted, type the code exactly as displayed (e.g., **WXYZ-1234**), then tap **Next** to continue to Google's account confirmation page. The rest of the sign-in flow is identical to the QR code path.

YouTube TV zAuth: Supported Devices and Requirements

Source: <https://sharvilgupta.mintlify.app/supported-devices>

See which TV platforms and streaming devices support YouTube's zAuth QR sign-in flow, plus the iPhone and Android requirements for scanning.

zAuth is supported across all major TV platforms and streaming devices that run the YouTube app. On the mobile side, iPhones running **iOS 11 or later** can scan QR codes natively using the built-in Camera app — no third-party scanner needed. Android phones are also supported through the YouTube app or by visiting youtube.com/activate. If your TV and phone meet the requirements below, you're ready to sign in with a single scan.

Supported TV Platforms

Android TV / Google TV

Supported on all devices running the YouTube app **v4.0 or later**. Includes NVIDIA SHIELD, Sony Bravia Google TV, Chromecast with Google TV, and similar Android-based sets.

Amazon Fire TV

Supported on Fire TV Stick, Fire TV Cube, and Fire TV Edition smart TVs running the YouTube app **v3.0 or later** from the Amazon Appstore.

Apple TV

Supported on Apple TV HD and Apple TV 4K running tvOS with the YouTube app **v3.0 or later** from the App Store.

Samsung Tizen TV

Supported on Samsung Smart TVs (2017 and newer) running Tizen OS with the YouTube app **v3.5 or later** from the Samsung Smart Hub.

LG webOS TV

Supported on LG Smart TVs (2017 and newer) running webOS 3.0 or later with the YouTube app **v3.5 or later** from the LG Content Store.

Chromecast with Google TV

Supported on all Chromecast with Google TV devices (HD and 4K) with the YouTube app **v4.0 or later** pre-installed.

iPhone Requirements

To scan the YouTube QR code using your iPhone, your device must meet the following minimum requirements:

Requirement	Minimum Version
iOS version	iOS 11 or later
Camera QR scanning	Built-in from iOS 11 (no third-party app needed)
Browser (for sign-in page)	Any modern version of Safari or Chrome
Google account	A valid Google account is required

For the best experience, keep your iPhone updated to the latest version of iOS. Older iOS 11 or 12 devices work but may load the Google sign-in page more slowly on some networks.

Android Phone as Secondary Device

Android phones also fully support the zAuth sign-in flow. You can authenticate in two ways:

- **YouTube app:** Open the YouTube app on your Android phone — if you're signed in, you may receive a push notification prompting you to approve the TV sign-in directly.
- **Manual activation:** Open any browser on your Android phone and visit `youtube.com/activate``, then enter the short code displayed beneath the QR code on your TV screen.

Not Supported

The zAuth QR sign-in flow is **not available** on the following:

- **Smart TVs without the YouTube app** — TVs running outdated firmware or manufacturers that have lost the YouTube app license cannot use zAuth.
- **Old Roku devices (prior to YouTube app v3.0)** — Very old Roku models that cannot be updated to YouTube app v3.0 or later do not support the QR flow. Use the manual code entry method at youtube.com/activate instead.
- **Web browsers on the TV itself** — Accessing YouTube through a TV's built-in web browser (rather than the dedicated YouTube app) does not trigger the zAuth QR flow; standard username/password sign-in is required in that context.

How to Switch or Add Google Accounts on YouTube on TV

Source: <https://sharvilgupta.mintlify.app/troubleshooting/account-switching>

How to switch between Google accounts on YouTube on TV, sign out of one account, and sign in with a different one using the QR code flow.

YouTube on TV supports signing in with one Google account at a time. If you want to switch to a different account — for example, from a family member's account to your own, or from a personal account to a YouTube Premium account — you'll need to sign out first and repeat the QR sign-in flow.

Signing Out of the Current Account

Open the Account menu

On the TV, navigate to your **YouTube profile icon** in the top-right corner of the screen using your remote.

Select Account

From the dropdown menu, select **Account**.

Choose Sign Out

Scroll down and choose **Sign Out** from the account options list.

Confirm sign-out

When prompted, confirm that you want to sign out. YouTube will return you to the signed-out home screen, where you'll see the **Sign In** option.

Signing In with a Different Account

Select Sign In

From the YouTube signed-out home screen, select **Sign In**.

Scan the new QR code

A new QR code is displayed on the TV. Open the Camera app on your iPhone and point it at the code.

Choose the desired Google account

After scanning, your iPhone will display a confirmation screen. Select the **Google account** you want to use for this TV session.

Tap Allow

Tap **Allow** to complete the authorisation. The TV will sign in to the new account automatically within a few seconds.

Signed In on iPhone with Multiple Accounts

If your iPhone has multiple Google accounts signed in — for example, through the Gmail app, Google app, or iOS account settings — you'll see **all of them listed** on the confirmation screen after scanning. Simply tap the account you want to use for the TV. You don't need to switch accounts on your iPhone beforehand.

Shared TV Use Cases

For households where multiple people use the same TV, we recommend using **YouTube Profiles** (available within a single Google account) rather than repeatedly switching Google accounts. YouTube Profiles give each household member their own recommendations, watch history, and playlists — without the friction of signing out and signing back in.

To learn more, visit [YouTube Help: Manage profiles](#).

Work vs Personal Accounts

If you use a **Google Workspace** account (formerly G Suite) provided by your employer or school, your organisation may have policies that restrict YouTube access on external or non-managed devices. This is controlled by your Google Workspace administrator and cannot be changed from the TV or your personal iPhone.

If you see an account restriction error when attempting to sign in with a Workspace account, contact your **IT administrator** to ask whether YouTube device authorisation can be permitted for your account. As an alternative, you can sign in with a personal Google account that is not managed by your organisation.

YouTube TV QR Code Sign-In: Frequently Asked Questions

Source: <https://sharvilgupta.mintlify.app/troubleshooting/faq>

Answers to the most common questions about YouTube on TV's QR code sign-in — covering setup, security, account management, and troubleshooting.

Here are the most frequently asked questions about signing in to YouTube on your TV using the QR code flow.

Do I need to be on the same Wi-Fi network as my TV?

No. The QR code sign-in flow does not require your iPhone and TV to be on the same network or to communicate with each other directly.

Here's how the flow works:

- The **TV** polls Google's authorisation servers waiting for approval.
- The **QR code** encodes a URL that opens a page on Google's servers (not on the TV).
- Your **iPhone** scans the code, opens that URL, and you approve the request — all via Google's servers.

Because both devices communicate **independently with Google**, your phone can be on **cellular data**, a different Wi-Fi network, or even a VPN, and sign-in will still work.

Can I use an Android phone instead of an iPhone?

Yes. The QR sign-in flow is not iPhone-exclusive. On Android, you have a few options:

- **Google Lens:** Open the Google Lens app (or tap the Lens icon in the Google Search widget) and point it at the QR code.
- **Camera app (Pixel and Samsung devices):** These devices support QR scanning natively through the stock Camera app, similar to iPhone.
- **YouTube app:** Open the YouTube app on your Android phone, tap your profile icon, and look for the **Watch on TV** option, which can initiate or link a TV session.
- **Manual entry:** Visit youtube.com/activate in any browser and type in the 8-character code shown beneath the QR code on your TV.

Is it safe to scan the QR code on a public or shared TV?

Only scan QR codes on TVs you trust. Here's what to consider:

- Once you scan and approve the code, the TV gains a persistent access token that allows it to use YouTube **as you** — accessing your watch history, subscriptions, recommendations, and (if applicable) your YouTube TV subscription.
- If someone else is present and watching when you scan, they could continue using the TV with your account after you leave.
- On hotel, gym, or other public TVs, **always sign out** before leaving by going to **Profile > Account > Sign Out**.
- You can also remotely revoke any TV session from myaccount.google.com/security under **Third-party apps & services**.

What happens if I lose my iPhone after signing in?

Your TV session continues independently — the iPhone is only needed during the initial sign-in step. Once the TV has received its access and refresh tokens from Google, it no longer communicates with your phone.

If your iPhone is lost or stolen, take the following steps:

1. From another device, go to myaccount.google.com/security.
2. Under **Third-party apps & services**, find the YouTube TV session and **revoke its access**.
3. The TV will be signed out on its next token refresh (usually within an hour), or immediately when it next tries to use the revoked token.

Your Google account credentials are never stored on the TV itself — only a token — so losing your phone does not expose your password.

How many TVs can I sign in to with one Google account?

Multiple TVs are supported. Google allows several concurrent active TV sessions per Google account. Each TV device completes its own independent authorisation flow and receives its own **refresh token**, so sessions don't interfere with each other.

You can review all active TV sessions and revoke individual ones at any time from myaccount.google.com/security > **Third-party apps & services**.

Can I sign in without scanning — just typing a code?

Yes. Every QR code displayed on your TV also has a corresponding **8-character alphanumeric code** shown in smaller text beneath the QR image.

To sign in manually:

1. On your phone, tablet, or computer, open a browser and go to youtube.com/activate.
2. Sign in with your Google account if prompted.
3. Type in the 8-character code exactly as shown on your TV screen.
4. Confirm the sign-in.

The TV will detect the approval within seconds. This method works on any device with a browser — no camera required.

The TV keeps showing me QR codes repeatedly even after I signed in

This usually means the **TV app isn't receiving the successful authorisation response** from Google, even though the sign-in was approved on your phone. The TV keeps regenerating new codes because it thinks the previous ones were never confirmed.

Steps to resolve:

1. **Check the TV's internet connection** — go to network settings and run a connectivity test. Even if other apps work, a restrictive firewall might be blocking oauth2.googleapis.com specifically.
2. **Restart the YouTube app** — press the Home button on your remote, close the YouTube app fully, and relaunch it.
3. **Clear the YouTube app cache** — on most smart TVs, go to **Settings > Apps > YouTube > Clear Cache**. This removes any corrupt session state that might be preventing the token from being saved.
4. **Restart the TV** — a full power-cycle (unplug for 30 seconds) resets the app environment.
5. If the issue persists, try signing in using the **manual code entry** method at youtube.com/activate, which uses a slightly different network path.

Does this work for YouTube TV (the live TV subscription) as well as regular YouTube?

Yes. The same QR code sign-in flow is used for both:

- The **free YouTube app** available on smart TVs, streaming sticks, and game consoles.
- The **YouTube TV app** (the live TV subscription service with broadcast and cable channels).

Both apps use the same OAuth 2.0 Device Authorization Grant flow and the same Google account infrastructure, so the sign-in experience is identical regardless of which service you're accessing.

Troubleshoot iPhone QR Code Scanning for YouTube TV

Source: <https://sharvilgupta.mintlify.app/troubleshooting/qr-code-not-scanning>

Troubleshoot iPhone camera issues preventing QR code scanning for YouTube TV sign-in, including iOS settings, lighting, and distance tips.

If your iPhone camera isn't recognizing the QR code on your TV screen, several common factors can interfere with the scan. This guide walks through every fix from most to least likely.

QR Code scanning is disabled in iPhone Settings

The iPhone Camera app has a dedicated toggle for QR code detection that must be enabled. To check:

1. Open the **Settings** app on your iPhone.
2. Scroll down and tap **Camera**.
3. Make sure the **Scan QR Codes** toggle is switched **on** (green).

> **Note:** This toggle is off by default on some older iOS versions. If you've never changed this setting, it's worth verifying even if your iPhone is relatively recent.

Camera app not in Photo mode

QR code detection only works when the Camera app is in standard **Photo** mode. Switching to Video, Portrait, Slo-Mo, or any other mode disables the QR scanner.

- Open the Camera app and swipe the mode selector at the bottom until **PHOTO** is highlighted.
- Point the camera at the QR code again and wait a moment for detection.

Phone too close or too far from the screen

Distance matters more than most people realise. The optimal range for scanning a QR code on a TV screen is **20–50 cm (roughly 8–20 inches)**.

- **Too close:** The QR code fills too much of the camera frame and the decoder can't locate the corner markers.
- **Too far:** The camera resolution isn't sufficient to resolve the individual pixels of the code.

Start at about arm's length and slowly move closer until a yellow box appears around the code.

Screen glare or reflections

Room lighting, windows, and overhead lights can create reflections on your TV screen that wash out the dark modules of the QR code, making it unreadable.

- **Tilt your phone slightly** — even a 10–15° angle change is often enough to move a reflection out of frame.
- **Dim nearby lights** temporarily while scanning.
- **Close blinds or curtains** if a window is reflecting directly onto the TV.

TV screen brightness too low

The QR code relies on high contrast between its dark squares and white background. If your TV's brightness or backlight is set low (common for night-time viewing modes or ambient modes), the camera may struggle to distinguish the pattern.

- Increase your TV's **brightness** and **backlight** to their maximum values temporarily.
- Disable any **ambient**, **eco**, or **night** display mode while signing in.
- Once sign-in is complete, you can restore your preferred display settings.

iPhone camera lens dirty

A smudged or dirty camera lens scatters light and reduces the sharpness needed for QR decoding — especially at the small module sizes displayed on a TV.

- Wipe the rear camera lens gently with a **clean, dry microfibre or soft cloth**.
- Avoid using paper towels or clothing, which can scratch the lens coating.
- Try scanning again immediately after cleaning.

QR code has expired

For security reasons, each QR code displayed on your TV is valid for **30 minutes**. After that, the device code in the QR payload is no longer accepted by Google's authorisation servers.

- If the code shows a **red countdown timer**, or your TV displays a **"Code expired"** message, the current code can no longer be scanned.
- **Wait a moment** — the TV will automatically request and display a fresh QR code.
- Scan the new code as soon as it appears to avoid it expiring again.

iOS version below 11

Native QR code scanning built into the iOS Camera app was introduced in **iOS 11**. Earlier versions do not support it at all.

- Go to **Settings > General > Software Update** to see your current iOS version and update if one is available.
- If your device cannot update beyond iOS 10 (e.g., iPhone 5 or earlier), you can still sign in using the **manual code entry** method:

1. On your TV, note the 8-character alphanumeric code displayed beneath the QR code.
2. On any browser, go to youtube.com/activate.
3. Enter the code and sign in with your Google account.

Prefer typing to scanning? You can always sign in without the camera by entering the code manually. See [Sign In by Entering a Code on Your iPhone](/sign-in/scan-with-iphone) for step-by-step instructions.

YouTube TV Sign-In Error Codes and How to Fix Them

Source: <https://sharvilgupta.mintlify.app/troubleshooting/sign-in-errors>

Reference guide for YouTube TV sign-in error messages including expired_token, access_denied, slow_down, network errors, and account issues.

When the sign-in flow encounters a problem, YouTube on TV displays an error message. This reference lists every error you might see, what it means, and the exact steps to resolve it. Where applicable, the underlying OAuth 2.0 error code (as defined in RFC 8628) is noted for developers.

What it means: Each QR code contains a device code with a **30-minute time-to-live (TTL)**. This error appears when that window elapsed before you completed the approval step on your iPhone.

Underlying OAuth 2.0 error: **expired_token** — the device code TTL defined in the original authorization request has passed. The TV must restart the flow by calling **POST /o/device/code** to obtain a new device code.

How to fix:

1. Stay on the TV screen — it will **automatically request a new device code** and display a fresh QR code within a few seconds.
2. Scan the new QR code with your iPhone promptly.
3. Tap **Allow** on the confirmation screen before the next 30-minute window closes.

> **Tip:** If you're repeatedly hitting this error, make sure your iPhone is nearby and signed in to the Google account you want to use before starting the flow on the TV.

What it means: You (or someone else) tapped **Deny** on the Google account confirmation screen on your iPhone. The authorisation grant was explicitly rejected, so no access token was issued.

Underlying OAuth 2.0 error: `access_denied` — the user declined the authorization request on the approval device. The current device code is permanently invalidated and cannot be reused.

How to fix:

1. On the TV, select **Sign In** to restart the flow and generate a new QR code.
2. Scan the new code with your iPhone.
3. On the confirmation screen, tap **Allow** (not Deny) to complete the authorisation.

> **Note:** If you didn't tap Deny yourself, check whether another app or person dismissed the notification. Only one confirmation attempt is allowed per device code.

What it means: This error indicates a **network-level failure** — the TV was unable to reach Google's authorisation servers (`oauth2.googleapis.com`) to poll for the token or exchange the device code.

How to fix:

1. **Check your TV's internet connection.** Navigate to your TV's network settings and run a connection test.
2. **Restart your router and TV.** Power-cycle both devices, then try again.
3. **Try a different DNS server.** On corporate or school networks, the default DNS may block Google services. Switch your TV's DNS to `8.8.8.8` (Google) or `1.1.1.1` (Cloudflare).
4. **Check for firewall restrictions.** Ensure that `oauth2.googleapis.com` and `accounts.google.com` are not blocked by a network firewall or content filter.
5. If the problem only occurs on one TV, try signing in on a different network to isolate whether the issue is with your home network or the TV itself.

What it means: The Google account you approved on your iPhone is subject to restrictions that prevent it from authorising third-party device sign-ins. This most commonly affects accounts managed by **Google Family Link** (child accounts under 13).

How to fix:

- **Use a standard, unmanaged Google account** to sign in to YouTube on TV.
- If the account is a Family Link child account, a **parent or guardian** can review and approve the connection from the Family Link app under **Controls > Apps**.
- If you're using a Google Workspace (school or work) account, your administrator may have restricted device authorisation — contact your IT department.

What it means: Google's authorisation servers apply **rate limiting** to device code polling requests. This error occurs when a TV (or network) has made too many rapid, repeated polling requests in a short period — typically from retrying sign-in multiple times in quick succession.

Underlying OAuth 2.0 errors: Two related error codes can trigger this condition:

- `slow_down` — the TV is polling `POST /token` too frequently. The polling interval must be increased (typically by at least 5 seconds) before retrying.
- `authorization_pending` — a normal intermediate state indicating the user has not yet approved the request on their iPhone. The TV should continue polling at the prescribed interval; no user action is needed.

How to fix:

1. **Wait at least 5 minutes** without attempting to sign in again.
2. After the waiting period, restart the sign-in flow on the TV and proceed normally.
3. Avoid pressing **Sign In** repeatedly or power-cycling the TV mid-flow, as each attempt generates new polling requests.

What it means: YouTube on TV maintains **one active authenticated session** at a time. A different Google account is currently signed in, and the TV rejected the new authorisation because it conflicts with the existing session.

How to fix:

1. **Sign out of the current account** on the TV:
 - Navigate to your YouTube profile icon (top-right corner).
 - Select **Account > Sign Out** and confirm.
2. Once signed out, select **Sign In** from the YouTube home screen.
3. Scan the new QR code with your iPhone and approve with the desired account.

What it means: The TV encountered a **DNS resolution failure or TLS handshake error** when trying to contact Google's servers. This is different from a general network error — the TV has internet access but cannot establish a trusted connection to Google specifically.

Common causes and fixes:

Cause	Fix
DNS server can't resolve google.com	Change TV DNS to 8.8.8.8 or 1.1.1.1
TV's system clock is wrong	Set date/time to automatic in TV settings — TLS certificates are validated against the current time
Router firmware issue	Restart your router; update router firmware if available
ISP-level DNS hijacking	Use a manual DNS address instead of the ISP-assigned one

> **Important:** TLS certificate validation **requires accurate system time**. If your TV's clock is set to a wrong year or date, every HTTPS connection to Google will fail. Always check TV date/time settings first.

If none of the fixes above resolve your issue, or if your error message isn't listed here, visit Google's account support at support.google.com/accounts for further assistance. You can also review your Google account's connected devices and revoke any unwanted sessions from myaccount.google.com/security.